

1º Trabalho de Projeto e Análise de Algoritmos

Escalonamento de Aulas via Transformar para Conquistar e Estratégia Gulosa

Gabriel Jared de Barros Amorim, Jonathan Santos Tadei,
Lucas Ivanov Costa, Vinicius Eduardo Moraes Oliveira

¹Colegiado de Ciência da Computação – Universidade Estadual do Oeste do Paraná (UNIOESTE)
Campus de Cascavel – PR – Brasil

Abstract. *This report compares two greedy solutions to the classroom-scheduling (interval partitioning) problem, under the Greedy Strategy and Transform-and-Conquer paradigms. The first solution (Greedy) applies the greedy room allocation directly on the original input order; the second (TC-Greedy) sorts classes by start time before applying the very same greedy core. We present both a theoretical (asymptotic cost) and an empirical (measured running time over 28 input sizes, from 10 to 1,000,000 classes) analysis, showing that although both solutions share the same $O(n^2)$ worst-case class, the sorting step yields a measurable practical gain beyond a certain input size.*

Resumo. *Este trabalho compara duas soluções gulosas para o problema de escalonamento de aulas em salas (minimização do número de salas), sob os paradigmas Estratégia Gulosa e Transformar para Conquistar. A primeira solução (Greedy) aplica a alocação gulosa diretamente sobre a ordem original de entrada; a segunda (TCGreedy) ordena as aulas por horário de início antes de aplicar o mesmo núcleo guloso. São apresentadas as análises teórica (polinômios de custo assintótico) e prática (tempos de execução medidos empiricamente sobre 28 tamanhos de entrada, de 10 a 1.000.000 aulas), mostrando que, apesar de ambas pertencerem à mesma classe de pior caso $O(n^2)$, a etapa de ordenação produz ganhos práticos mensuráveis a partir de um determinado tamanho de entrada.*

1. Introdução e Descrição do Problema

O problema consiste em alocar n aulas, cada uma com um horário de início e término, ao menor número possível de salas, respeitando as seguintes restrições:

- Duas aulas não podem ocupar a mesma sala no mesmo instante;
- Não é necessário intervalo entre duas aulas consecutivas na mesma sala – uma aula pode começar exatamente no instante em que a anterior termina.

Foram implementadas duas soluções gulosas que compartilham o mesmo núcleo de alocação, diferindo apenas na etapa de pré-processamento da entrada:

- **Greedy:** processa as aulas na ordem original do arquivo de entrada;
- **TCGreedy:** aplica a etapa de “Transformar” – ordena as aulas por horário de início, de forma crescente – e só então aplica o mesmo núcleo guloso.

Este problema é um caso clássico da literatura de algoritmos: corresponde exatamente ao Exercício 16.1-4 de [Cormen et al. 2009], apresentado logo após o problema de seleção de atividades (Seção 16.1) como o problema de *escalonar atividades usando o menor número possível de salas de conferência*. O próprio livro observa que esse problema equivale à coloração de um grafo de intervalos (cada aula é um vértice, e duas aulas incompatíveis – que se sobrepõem – são ligadas por uma aresta): o número mínimo de salas necessárias é igual ao número cromático desse grafo, que por sua vez é igual à *profundidade máxima* de sobreposição simultânea de aulas (o maior número de aulas ativas ao mesmo tempo, em qualquer instante). O mesmo problema – com a mesma função de seleção usada pelo TCGreedy, “selecionar a aula com o menor tempo de início” – também é apresentado no material da disciplina, na seção “Escalonamento de Tarefas” [Brun 2026].

2. Descrição das Soluções

2.1. Núcleo Guloso Compartilhado

O núcleo, doravante chamado `alocarSalas`, mantém uma lista de salas abertas, cada uma com o horário de término da última aula alocada nela. Para cada aula, na ordem em que é processada:

1. Percorre as salas já abertas em busca da primeira cujo horário de término seja menor ou igual ao horário de início da aula atual (*first-fit*);
2. Se encontrar, reutiliza essa sala, atualizando seu horário de término;
3. Caso contrário, abre uma nova sala para a aula atual.

```
alocarSalas(aulas):  
    terminoDaSala = []  
    para cada aula em aulas:  
        sala = procurar em terminoDaSala uma sala com  
                termino <= aula.inicio  
        se encontrou:  
            atualizar termino dessa sala para aula.termino  
        senao:  
            abrir nova sala com termino = aula.termino  
    retornar numero de salas abertas
```

2.2. O Paradigma Transformar para Conquistar

Transformar para Conquistar (*Transform-and-Conquer*) é uma técnica de projeto de algoritmos que resolve um problema em duas etapas [Levitin 2011]: primeiro **transforma-se** a instância de entrada em uma forma mais conveniente – sem alterar o problema em si – e só então **conquista-se** (resolve-se) essa instância transformada, usando um algoritmo que se torna mais simples, mais rápido ou mais eficaz justamente por operar sobre a versão transformada em vez da original. Levitin [Levitin 2011] classifica essa técnica em três categorias, conforme a natureza da transformação:

1. **Simplificação de instância:** transformar a entrada em uma instância mais simples ou mais conveniente do *mesmo* problema – o exemplo clássico é a *pré-ordenação* (*presorting*), em que ordenar a entrada de antemão simplifica um algoritmo que viria a seguir;

2. **Mudança de representação:** transformar a entrada para uma representação diferente da mesma instância (ex.: estruturas de dados balanceadas);
3. **Redução a outro problema:** transformar a instância em uma instância de um problema *diferente*, para o qual já existe um algoritmo conhecido.

O TCGreedy deste trabalho é um exemplo direto de **simplificação de instância por pré-ordenação**: a etapa “Transformar” ordena as aulas por horário de início antes de qualquer alocação, e a etapa “Conquistar” aplica o mesmo núcleo guloso (`alocarSalas`) usado pelo Greedy, sem nenhuma modificação. O ganho de pré-ordenar não está em mudar o algoritmo que resolve o problema, mas em entregar a ele uma entrada em uma ordem que o torna mais eficaz – é exatamente esse o espírito da técnica: a etapa de “Conquistar” nem precisa saber que a entrada foi transformada, apenas se beneficia dela.

2.3. Greedy

Aplica `alocarSalas` diretamente sobre o vetor de entrada, sem qualquer ordenação prévia – ou seja, sem a etapa de “Transformar”.

2.4. TCGreedy

Ordena o vetor de aulas por horário de início ($O(n \log n)$) – a etapa “Transformar” descrita na Seção 2.2 – e em seguida aplica a **mesma** função `alocarSalas` usada pelo Greedy – a etapa “Conquistar”. A única diferença entre as duas soluções está, portanto, na ordem em que as aulas chegam ao núcleo guloso. Essa combinação de ordenação por início seguida de alocação gulosa corresponde exatamente ao algoritmo de escalonamento apresentado em aula [Brun 2026], cuja função de seleção é “selecionar a tarefa com o menor tempo de início”.

3. Análise de Complexidade Assintótica

3.1. Modelo de custo

Seguindo o método de contagem de operações básicas apresentado em aula (atribuição, teste lógico, incremento, indexação, soma e retorno como operações de custo unitário; o tempo de uma sequência de comandos é o maior tempo de qualquer comando da sequência; o tempo de comandos aninhados é obtido pelo produto/soma das complexidades), reescrevemos o núcleo `alocarSalas` (Seção 2.1) com laços indexados, equivalentes ao `for-each` usado na implementação em Java:

```
1   terminoDaSala = novo vetor de tamanho n
2   numSalas = 0
3   operacoes = 0
4   for (i = 0; i < n; i++)
5       salaEscolhida = -1
6       for (s = 0; s < numSalas; s++)
7           operacoes++
8           if (terminoDaSala[s] <= aulas[i].inicio)
9               salaEscolhida = s
10          break
```

```

11      if (salaEscolhida == -1)
12          salaEscolhida = numSalas
13          numSalas++
14      terminoDaSala[salaEscolhida] = aulas[i].termino
15  retornar numSalas, operacoes

```

3.2. Núcleo guloso – pior caso (comum às duas soluções)

O pior caso ocorre quando a condição da linha 8 nunca é verdadeira – por exemplo, quando as n aulas se sobrepõem todas mutuamente (compartilham um mesmo instante em comum). Nesse cenário, o laço interno nunca encontra sala livre: roda até o fim sem `break` (linhas 9–10 nunca executam) e o `if` da linha 11 é sempre verdadeiro (sempre abre sala nova). Quando o laço da linha 6 começa a executar para a i -ésima aula ($i = 0, \dots, n-1$), o valor de `numSalas` naquele momento é exatamente i , pois uma sala nova foi aberta em cada uma das i iterações anteriores. Isso caracteriza um **laço aninhado dependente**: o limite do laço interno cresce junto com o índice do laço externo, de forma análoga ao exemplo de laços dependentes visto em aula – e por isso seu custo total é obtido somando uma progressão aritmética.

A Tabela 1 detalha o custo de cada linha.

Tabela 1. Custo do núcleo `alocarSalas` no pior caso, linha a linha.

Linha	Custo/execução	Nº de execuções	Total
1	n (alocação/zeragem)	1	n
2	1 atrib.	1	1
3	1 atrib.	1	1
4	1 atrib. + $n(\text{tlog} + \text{incr}) + \text{tlog}$	1	$2n + 2$
5	1 atrib.	n vezes	n
6	1 atrib. + $i(\text{tlog} + \text{incr}) + \text{tlog}$	$\sum_{i=0}^{n-1}, \text{numSalas} = i$	$n^2 + n$
7	1 incr.	$\sum_{i=0}^{n-1} i$	$\frac{n^2 - n}{2}$
8	1 index + 1 tlog	$\sum_{i=0}^{n-1} i$	$n^2 - n$
9–10	–	pior caso: nunca executa	0
11	1 tlog	n vezes	n
12	1 atrib.	n vezes (sempre verd.)	n
13	1 incr.	n vezes (sempre verd.)	n
14	1 index + 1 atrib.	n vezes	$2n$
15	1 retorno	1	$O(1)$

Note-se que a linha 8 acessa o vetor `terminoDaSala[s]` dentro do teste lógico – por isso soma-se 1 indexação a cada execução, seguindo exatamente o mesmo padrão do exemplo de aula “`if (A < Vetor[i]) → 1 index + 1 teste log = 2`”. As linhas 7 e 13 usam o operador de incremento (`operacoes++`, `numSalas++`), tal como implementado no código Java – custando 1 operação cada, o mesmo tratamento dado ao A++ do exemplo de laços dependentes visto em aula.

Somando todas as linhas:

$$T_{\text{nucleo}}(n) = n + 1 + 1 + (2n + 2) + n + (n^2 + n) + \frac{n^2 - n}{2} + (n^2 - n) + n + n + n + 2n + O(1) \quad (1)$$

$$T_{nucleo}(n) = \frac{5n^2}{2} + \frac{17n}{2} + O(1) \in \Theta(n^2) \quad (2)$$

ou seja, $O(n^2)$ no pior caso.

3.3. Greedy

O Greedy aplica apenas o núcleo, diretamente sobre a entrada, sem custo adicional:

$$T_{Greedy}(n) = T_{nucleo}(n) = \frac{5n^2}{2} + \frac{17n}{2} + O(1) \in O(n^2) \quad (3)$$

3.4. TCGreedy

O TCGreedy soma ao núcleo o custo da ordenação por horário de início, tratada como uma chamada de função de complexidade conhecida (assim como funções puras vistas em aula, ex. *raiz(n)*, *potencia(base, expoente)*). O método `Arrays.sort` usado para ordenar um vetor de objetos (como `Aula[]`) implementa uma variante estável de *merge sort* (TimSort), cujo número de comparações no pior caso é $T_{sort}(n) = n \log_2 n$. Logo:

$$T_{TCGreedy}(n) = T_{sort}(n) + T_{nucleo}(n) = n \log_2 n + \frac{5n^2}{2} + \frac{17n}{2} + O(1) \in O(n^2) \quad (4)$$

pois o termo quadrático domina o termo $n \log n$ para n suficientemente grande.

3.5. Consequência teórica

As duas soluções pertencem à mesma classe assintótica de pior caso, $O(n^2)$, já que compartilham o mesmo núcleo de alocação – o termo $n \log n$ adicional do TCGreedy é absorvido pelo termo quadrático que domina em ambos os polinômios. A diferença entre as soluções não aparece na classe assintótica de pior caso, mas sim (a) no comportamento médio prático e (b) na qualidade da solução (número de salas), como discutido na Seção 5.

4. Metodologia Experimental

4.1. Configuração da Máquina de Testes

- **Sistema operacional:** Arch Linux, kernel 7.1.3-arch1-3;
- **Processador:** Intel(R) Core(TM) i5-8265U CPU @ 1.60GHz (4 núcleos / 8 threads);
- **Memória RAM:** 7,6 GiB;
- **Linguagem/runtime:** Java (OpenJDK 21.0.11);
- **IDE:** nenhuma – compilação e execução via linha de comando (terminal), sem uso de ambiente de desenvolvimento integrado;
- **Compilação:** `javac`, sem *flags* de otimização especiais;
- **Execução:** `java -cp out BenchmarkRunner <pasta-entradas> <csv-saida> <timeout>`.

4.2. Processo de Coleta dos Tempos

Para cada um dos tamanhos de entrada disponibilizados e para cada estratégia:

1. A entrada é lida uma única vez – a leitura do arquivo **não** entra na medição de tempo;
2. O núcleo guloso é executado 6 vezes sobre os mesmos dados em memória, cronometrado com `System.nanoTime()`;
3. A primeira execução é descartada (aquecimento de JIT);
4. Calcula-se a média das 5 execuções restantes;
5. São também registrados, por execução, o número de salas resultante e o número de operações de comparação do núcleo guloso (variáveis de interesse);
6. O `BenchmarkRunner` suporta um orçamento de tempo opcional por execução (parametrizável ou totalmente desligável), pensado para evitar que uma entrada muito grande trave o benchmark indefinidamente. A coleta final apresentada neste relatório foi feita com esse orçamento **desligado**, para obter tempos reais mesmo nos tamanhos maiores – o maior tempo médio observado foi o do Greedy em $n = 1.000.000$, $\sim 121,1$ s por execução.

Os dados brutos estão disponíveis em `resultados/tempos.csv`, no repositório de código-fonte que acompanha este relatório.

5. Resultados

A Figura 1 mostra o tempo médio de execução em função do tamanho da entrada, em escala linear; a Figura 2 mostra os mesmos dados em escala log-log, que evidencia melhor o comportamento nas ordens de grandeza menores. A Figura 3 mostra o número médio de operações de comparação do núcleo guloso (a variável de interesse contada pela instrumentação do `BenchmarkRunner`) em função de n , também em escala log-log – diferente do tempo de parede, essa métrica não é afetada pelo custo do *sort* nem por ruído da JVM, sendo um indicador mais direto do comportamento assintótico do núcleo. A Tabela 2 resume os valores medidos para tamanhos de entrada representativos.

Principais observações:

- **Número de salas** cresce de forma aproximadamente linear com n em ambas as soluções (ex.: $n = 250.000 \rightarrow 125.691$ salas no Greedy, 124.877 no TCGreedy; $n = 1.000.000 \rightarrow 642.836$ no Greedy, 642.819 no TCGreedy), confirmando que o núcleo guloso de fato se comporta como $O(n^2)$ na prática, não apenas no pior caso teórico, já que o número de salas abertas r é proporcional a n ;
- **TCGreedy realiza menos operações** de comparação no núcleo guloso do que Greedy em **todos** os tamanhos testados (ex.: $n = 100.000 \rightarrow 2,51 \times 10^9$ operações no Greedy contra $2,18 \times 10^9$ no TCGreedy; $n = 1.000.000 \rightarrow 3,21 \times 10^{11}$ no Greedy contra $2,48 \times 10^{11}$ no TCGreedy – uma redução de $\sim 23\%$), e resulta em **menos ou o mesmo número de salas** (qualidade de solução igual ou melhor) – a ordenação por início ajuda o *first-fit* a encontrar salas livres mais cedo, em média, e evita alocações desnecessárias. A Figura 3 mostra que essa vantagem se mantém proporcional ao longo de todas as ordens de grandeza testadas, com as duas curvas paralelas em escala log-log (inclinação ≈ 2 , confirmando o comportamento $O(n^2)$ do núcleo em ambas as soluções);

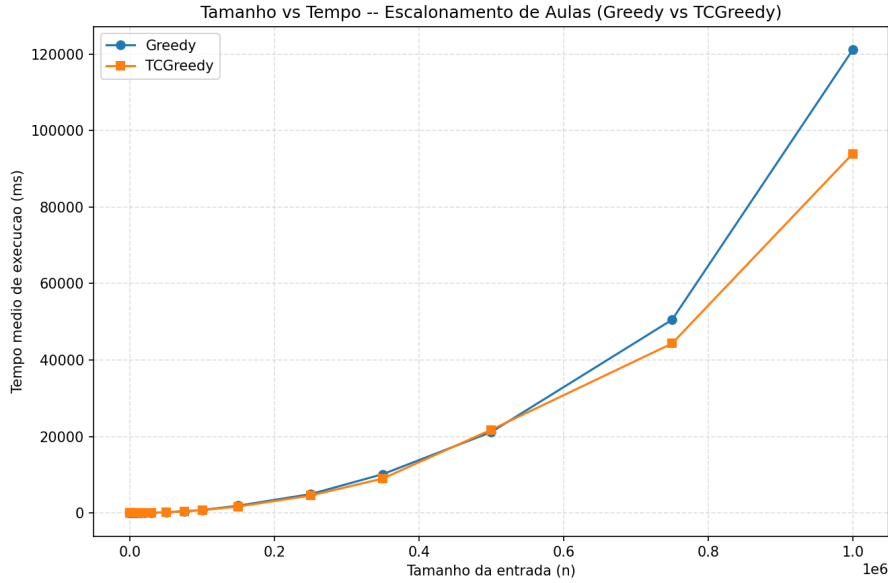


Figura 1. Tamanho vs. tempo médio de execução (ms) – escala linear.

- **Ponto de cruzamento no tempo de parede:** para n pequeno (até ~ 20.000), o Greedy é mais rápido em tempo de parede, pois o custo constante do `Arrays.sort` supera a economia de operações do núcleo (ex.: $n = 20.000 \rightarrow$ Greedy 26,5ms vs TCGreedy 35,6ms). A partir de $n \approx 30.000$, esse quadro se inverte – o TCGreedy passa a ser mais rápido na maioria dos tamanhos maiores (ex.: $n = 30.000 \rightarrow$ Greedy 65,9ms vs TCGreedy 65,5ms; $n = 1.000.000 \rightarrow$ Greedy 121.092,9ms vs TCGreedy 93.954,9ms), pois o custo $O(n \log n)$ do `sort` passa a ser irrelevante frente à economia de operações $O(n^2)$ do núcleo. Há uma inversão pontual em $n = 500.000$ (Greedy 21.163,5ms vs TCGreedy 21.774,8ms), provavelmente causada por ruído de medição (pausas de *garbage collector* ou variação de carga do sistema) e não por uma mudança real de comportamento algorítmico, já que a tendência volta a favorecer o TCGreedy nos dois tamanhos seguintes.

6. Análise Crítica

As duas soluções compartilham exatamente o mesmo núcleo guloso e, por isso, pertencem à mesma classe assintótica de pior caso, $O(n^2)$ – a etapa de “Transformar” (ordenação, $O(n \log n)$) não muda essa classe, pois o núcleo domina o crescimento assintótico. Isso ficou evidente na prática: o número de operações e o número de salas crescem de forma consistente com n^2 nas duas soluções.

Uma verificação quantitativa reforça esse ponto: substituindo $n = 1.000.000$ na fórmula do pior caso teórico obtida na Seção 3, $T_{nucleo}(1.000.000) = \frac{5}{2} \times (10^6)^2 + \frac{17}{2} \times 10^6 + O(1) \approx 2,50 \times 10^{12}$ operações – mas o valor efetivamente medido para o Greedy nesse tamanho foi de apenas $3,21 \times 10^{11}$ operações (Tabela 2), cerca de 13% do teto teórico, um fator de $\sim 8\times$ de folga. Essa diferença não invalida a análise: a fórmula $\frac{5n^2}{2} + \frac{17n}{2}$ foi derivada para o pior caso absoluto, em que as n aulas se sobrepõem **todas** mutuamente e o laço interno nunca encontra sala livre (`numSalas` cresce até n a cada

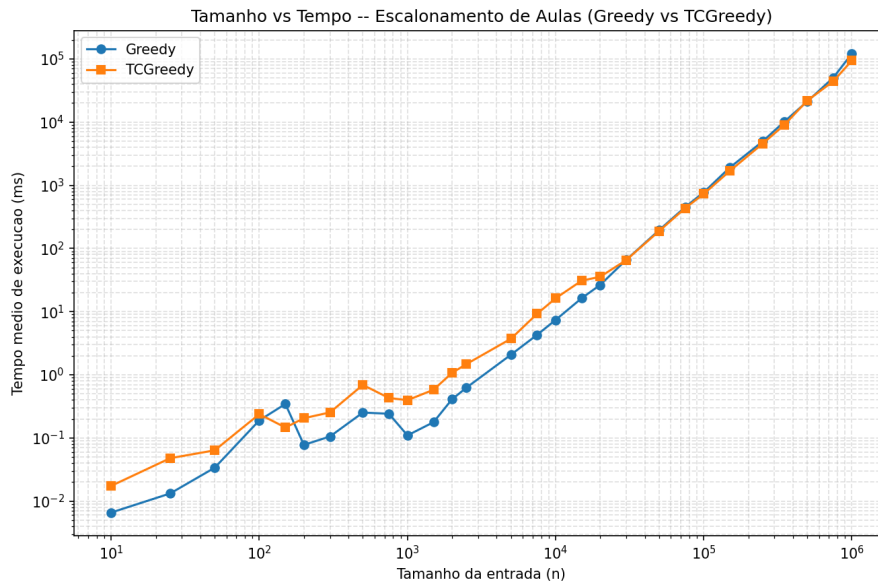


Figura 2. Tamanho vs. tempo médio de execução (ms) – escala log-log.

aula processada). Os dados reais de entrada têm sobreposição substancial – suficiente para produzir um crescimento genuinamente quadrático, como evidenciado pela inclinação ≈ 2 da Figura 3 em escala log-log – mas não a adversidade máxima: o número final de salas em $n = 1.000.000$ chega a apenas 642.836 ($\sim 64\%$ de n), não a n salas. Como consequência, uma fração relevante das varreduras do *first-fit* encontra uma sala livre antes de percorrer toda a lista de salas abertas, o que reduz a constante efetiva observada sem alterar a ordem de crescimento assintótica.

Ainda assim, o “Transformar” tem um efeito prático real e mensurável, mesmo sem mudar a classe assintótica de pior caso:

- Ele **reduz a constante multiplicativa** do núcleo $O(n^2)$ (menos operações por aula, em média, porque salas livres são encontradas mais cedo);
- Ele **melhora a qualidade da solução** (menos salas usadas) – um benefício que nem chega a ser exigido nos critérios formais deste trabalho, mas é um efeito colateral positivo interessante do pré-processamento;
- Ele **adiciona um custo fixo** (o *sort* em si), que só compensa a partir de um tamanho de entrada onde a economia de operações do núcleo supera esse custo – daí o ponto de cruzamento observado por volta de $n \approx 20.000$ –30.000.

A melhora na qualidade da solução (item anterior) não é uma coincidência empírica, mas consequência direta da teoria de coloração de grafos de intervalos discutida na Seção 3 [Cormen et al. 2009]: o número mínimo de salas necessárias é igual à profundidade máxima de sobreposição simultânea de aulas, e processar as aulas em ordem crescente de horário de início, alocando cada uma à primeira sala livre disponível – exatamente o que o TCGreedy faz – é conhecido por atingir esse mínimo teórico. Já o Greedy processa as aulas na ordem arbitrária do arquivo de entrada, sem essa garantia: uma aula que começa cedo mas aparece tarde no arquivo pode ser processada depois de aulas que começam mais tarde, fazendo com que o algoritmo abra salas que uma aula

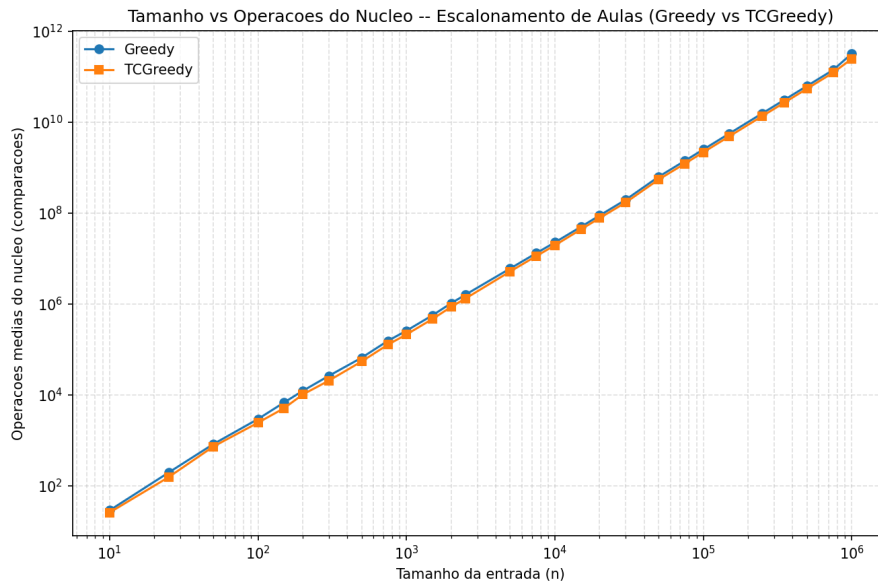


Figura 3. Tamanho vs. número médio de operações de comparação do núcleo – escala log-log.

ainda não processada poderia ter reaproveitado. É por isso que o TCGreedy iguala ou usa menos salas que o Greedy em **todos** os tamanhos testados, nunca o contrário.

Esse resultado ilustra bem o princípio geral do paradigma Transformar para Conquistar [Levitin 2011]: pré-processar a entrada não necessariamente melhora a complexidade assintótica de pior caso do algoritmo seguinte, mas pode melhorar substancialmente o comportamento médio/prático – e o ganho só se manifesta quando a etapa de “Conquistar” de fato tira proveito da transformação (aqui, o *first-fit* tira proveito de estar processando aulas em ordem crescente de início, mesmo sem ter sido redesenhado para isso).

Os tempos observados em $n = 1.000.000$ (a maior entrada testada – Greedy 121,09s, TCGreedy 93,95s por execução) reforçam, na prática, por que algoritmos $O(n^2)$ [Cormen et al. 2009] são considerados proibitivos para entradas grandes: cada execução isolada já leva mais de um minuto, mesmo em um processador moderno.

7. Conclusão

O experimento mostrou que, embora Greedy e TCGreedy compartilhem o mesmo núcleo de alocação e por isso tenham a mesma classe assintótica de pior caso ($O(n^2)$), a etapa de pré-ordenação do TCGreedy tem impacto prático real: reduz o número de operações do núcleo e iguala ou reduz o número de salas usadas em todos os tamanhos testados, e passa a compensar seu próprio custo (o *sort*) a partir de $n \approx 20.000$ – 30.000 , tornando-se a solução mais rápida em tempo de parede na maior parte dos tamanhos maiores testados (com uma única inversão pontual em $n = 500.000$, provavelmente ruído de medição). Isso confirma que o valor do paradigma Transformar para Conquistar não está necessariamente em mudar a ordem de complexidade assintótica, mas em explorar uma estrutura na entrada (aqui, a ordem cronológica das aulas) que barateia o comportamento médio do algoritmo que vem em seguida. Os tempos crescentes nas entradas maiores testadas (chegando a mais de um minuto por execução em $n = 1.000.000$) também evidenciaram, de forma

Tabela 2. Tempo médio de execução (ms), número de salas usadas e número médio de operações do núcleo, para tamanhos de entrada representativos (todos os valores são média de 5 execuções, sem interrupção por tempo).

n	Tempo médio (ms)		Salas usadas		Operações do núcleo	
	Greedy	TCGreedy	Greedy	TCGreedy	Greedy	TCGreedy
10	0,007	0,017	7	7	$2,90 \times 10^1$	$2,60 \times 10^1$
100	0,190	0,238	62	53	$2,96 \times 10^3$	$2,46 \times 10^3$
1.000	0,110	0,396	531	481	$2,59 \times 10^5$	$2,14 \times 10^5$
10.000	7,373	16,222	4.576	4.405	$2,27 \times 10^7$	$1,96 \times 10^7$
20.000	26,452	35,551	9.703	9.606	$8,86 \times 10^7$	$7,77 \times 10^7$
30.000	65,945	65,458	13.234	12.864	$1,97 \times 10^8$	$1,71 \times 10^8$
50.000	194,644	188,233	25.313	25.090	$6,30 \times 10^8$	$5,47 \times 10^8$
100.000	770,891	727,892	50.402	49.979	$2,51 \times 10^9$	$2,18 \times 10^9$
150.000	1.905,461	1.681,344	75.314	74.868	$5,64 \times 10^9$	$4,90 \times 10^9$
250.000	4.934,210	4.551,336	125.691	124.877	$1,57 \times 10^{10}$	$1,36 \times 10^{10}$
350.000	10.156,943	9.018,185	175.876	175.219	$3,07 \times 10^{10}$	$2,67 \times 10^{10}$
500.000	21.163,517	21.774,828	251.179	250.175	$6,27 \times 10^{10}$	$5,46 \times 10^{10}$
750.000	50.509,619	44.313,597	376.653	375.352	$1,41 \times 10^{11}$	$1,23 \times 10^{11}$
1.000.000	121.092,902	93.954,894	642.836	642.819	$3,21 \times 10^{11}$	$2,48 \times 10^{11}$

concreta, por que algoritmos $O(n^2)$ se tornam inviáveis à medida que n cresce.

8. Código-fonte

O código-fonte completo está disponível na pasta `src/` do repositório deste trabalho, incluindo os comandos de medição de tempo (`System.nanoTime()`) e contagem de operações (`EscalonadorGuloso.Resultado.operacoes()`).

Referências

- Brun, A. L. (2026). Estratégia gulosa (slides de aula – projeto e análise de algoritmos). Material didático, Colegiado de Ciência da Computação, UNIOESTE.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. MIT Press, 3rd edition.
- Levitin, A. (2011). *Introduction to the Design and Analysis of Algorithms*. Pearson, 3rd edition.